APIM: Documento Técnico Consolidado

Modernización de Infraestructura API - Banco Galicia

Versión: 1.0
Fecha: Enero 2026
Enfoque: Técnico y Arquitectónico
Author: fernando.l.gonzalez@bancogalicia.com.ar

Tabla de Contenidos

1. Contexto y Situación Actual
2. Camino Transitado: Evolución del Proyecto
3. Lecciones Aprendidas
4. Decisiones Técnicas y Arquitectónicas
5. Arquitectura Objetivo
6. Evaluación de Proveedores
7. Pasos Necesarios: Roadmap Técnico
8. Riesgos y Mitigaciones
9. Conclusiones y Próximos Pasos

1. Contexto y Situación Actual

1.1 Infraestructura Actual (3scale)

Escala de Operación:

- **Cluster monolítico:** +100 nodos, +10,000 pods, +600 namespaces
- **Volumen de tráfico:** ~8 mil millones de requests/mes
  - **East-West (interno):** 7.5B requests/mes (80%)
  - **North-South (externo):** 500M requests/mes (20%)
- **APIs en producción:** ~2,200 APIs
  - ~1,500 servicios internos reales
  - ~500 servicios batch (sin APIs)
  - ~200 external API facades/BFF

Enfoque Actual de Gestión de APIs:

El enfoque actual trata todas las APIs como si fueran APIs externas, cuando en realidad el **90% son de consumo interno**. Esto genera overhead innecesario y complejidad operativa para tráfico service-to-service que debería ser más directo y eficiente.

Arquitectura de Red:

- **Stretched network:** Red extendida para interconectar los dos sitios (PGA/CMZ)
- **Flujo de tráfico para APIs :**
  - **Expuestas a Internet:** F5/Fortinet (WAF, FW) → OCP DMZ (HAProxy → proxies reversos) → FW → F5 LB → OCP Prod (HAProxy) → 3scale Apicast → aplicaciones
  - **Consumidas internamente:** Servicio A (Namespace 1) → Sale del cluster → Load balancer (F5) → Re-entra al cluster (HAProxy) → 3scale Apicast → Servicio B (Namespace 2)
- **Hair-pinning pattern:** Servicios internos salen del cluster → load balancer → re-entran, generando latencia adicional innecesaria
- **Network policies:** Fuerzan todo el tráfico cross-namespace a través de 3scale

Alta Disponibilidad y Disaster Recovery:

- **Dos instancias independientes:** Una por sitio (Plaza/Matriz)
- **Bases de datos separadas:** Sin almacenamiento compartido

- **Sincronización manual:** Vía pipelines y scripts de automatización
- **Elementos sincronizados:**
  - Productos API y contratos
  - Certificados cliente (proceso manual)
  - Políticas de rewrite de headers
  - Backend endpoints y configuraciones
  - Credenciales de aplicación y API keys
  - ~2,500 aplicaciones con múltiples endpoints cada una

Limitaciones Críticas de DR:

- Solo es posible un esquema activo/standby, ya que se requiere volcar el tráfico completo del cluster a una u otra instancia; no es posible operar en modo activo-activo ni realizar switcheo selectivo de APIs o namespaces.
- Sin capacidad declarativa (no se puede convertir a CRDs)
- Comparación manual requerida entre sitios usando toolkit
- Sincronización basada en pipelines (export/import de configuraciones)
- Riesgo de desincronización cuando el pipeline falla parcialmente
- No hay failover automatizado (requiere cambio manual de DNS)

Modelo de Autenticación:

- **API Key Authentication (problemático):**
  - Headers básicos: API keys/client IDs
  - Tokens estáticos sin expiración
  - Revocación difícil - considerado anti-patrón de seguridad
  - Sin implementación OAuth2
  - Consumo interno: JWT básico simplificado
- **Dependencia crítica de DBs externas:** Tanto Redis como la base transaccional son single points of failure

Integración con Core Banking, Legacy y Terceros:

- Alta dependencia del core bancario mainframe a traves de CIS servicers
- Integración relevante con Oracle Service Bus y VMs legacy
- Mayoría de transacciones requieren múltiples hits a APIs bancarias
- 3scale mapea APIs externas como internas de OpenShift
- Las APIs de terceros también generan un volumen considerable de tráfico y deben ser tenidas en cuenta

Procesos de Publicación y Suscripción:

Los procesos de publicación de APIs y suscripción de aplicaciones son **complejos, lentos y de gran carga operativa**:

- **Publicación de APIs:** Proceso manual que requiere múltiples pasos, configuración en base de datos, sincronización entre sitios, y validación manual
- **Suscripción de aplicaciones:** Requiere creación manual de credenciales, configuración de políticas, y sincronización entre instancias de 3scale
- **Mantenimiento:** Actualización de configuraciones requiere procesos manuales, comparación entre sitios, y riesgo de desincronización
- **Carga operativa:** ~2,500 aplicaciones con múltiples endpoints requieren mantenimiento continuo y procesos manuales repetitivos
- **Falta de trazabilidad:** No hay capacidad declarativa ni GitOps, todo depende de procesos manuales y pipelines complejos

1.2 Pain Points Identificados

Técnicos:

- **Hair-pinning:** Crea latencia adicional de 25-50ms por salto y bottlenecks
- **Observabilidad rota y pobre:**
  - 3scale actúa como "firewall", lo que corta los traces completos
  - Falta de métricas avanzadas de rendimiento y trazabilidad end-to-end
- **Límite de escalabilidad:** 500 routes vs 2,200 APIs actuales; Apicast tarda mucho en iniciar debido a que el reload es completo y no dinámico.
- **Performance bottlenecks:** Con arquitectura actual
- **Sin capacidad declarativa:** Todo en base de datos, no GitOps

Operacionales:

- **Sincronización manual de DR:** Carga operativa compleja
- **Riesgo de drift de configuración:** Entre sitios
- **Failover manual:** Requiere intervención manual de DNS

- **Gestión de secrets compleja:** A través de instancias independientes
- **Overhead operativo:** Mantener dos sistemas separados

Seguridad:

- **Tokens estáticos:** Sin expiración, difícil revocación, facil de compartir
- **Anti-patrón regulatorio:** Observaciones en auditorías
- **Single point of failure:** Redis crítico

Timeline Crítico:

- **3scale End-of-Life:** Mediados 2027
- **Decisión requerida:** Q2 2026
- **Go-live objetivo:** Q4 2026

2. Camino Transitado: Evolución del Proyecto

2.1 Fase 1: Evaluación Inicial (Reemplazo Simple de APIM)

Período: Agosto - Octubre 2025

Objetivo: Migración directa de 3scale a un API Management alternativo, enfocado en paridad funcional y mínima disrupción.

Contexto:

- Deadline crítico: 3scale End-of-Life mediados 2027
- Necesidad de mantener continuidad operativa sin disrupciones
- Enfoque conservador: buscar solución que replique funcionalidades actuales
- Prioridad en herramientas de migración asistida para minimizar esfuerzo

Actividades realizadas:

- Evaluación de vendors con capacidades de API Management tradicional completas
- Prioridad en migración asistida y continuidad operativa
- Búsqueda de solución drop-in replacement
- Validación de compatibilidad con arquitectura OpenShift existente
- Análisis de modelos de pricing y costos operativos

Vendors evaluados en esta fase:

- **Red Hat Connectivity Link:**
  - Continuidad con ecosistema Red Hat/3scale
  - Herramientas de migración desde 3scale (tasa de éxito 6-7/10)
  - Integración nativa con OpenShift
  - Descuento 50% primer año para migraciones
- **Apigee (Google Cloud):**
  - Enterprise ready con amplia base de clientes
  - Capacidades avanzadas de API Management
  - Backing de Google
  - Incluso en el modelo on-premise ("hybrid"), era necesario enrutar tráfico a través de GCP, lo cual plantea un problema ya que no existen fundaciones ni presencia corporativa en esa nube pública
- **IBM API Connect:**
  - Solución legacy con experiencia en entornos bancarios
  - Evaluación rápida, descartado por arquitectura no cloud-native

Insight crítico descubierto: Las evaluaciones profundas de proveedores revelaron limitaciones arquitectónicas fundamentales que exigieron una visión estratégica más amplia. Se identificó que:

- Las soluciones tradicionales de API Management no optimizan para tráfico interno masivo
- El modelo de pricing por API call es inviable para 7.5B requests/mes internos
- La arquitectura actual con hair-pinning requiere rediseño, no solo reemplazo

2.2 Fase 2: Despertar Arquitectónico (Prioridad en API Gateway)

Período: Octubre - Noviembre 2025

Objetivo: Identificar solución de API Gateway que optimice tráfico interno masivo (7.5B requests/mes).

Contexto:
Las demostraciones de proveedores evidenciaron que la necesidad principal era la gestión eficiente del tráfico, más que el overhead tradicional de API Management.

Análisis realizado:

- Desglose detallado de patrones de tráfico: 7.5B requests/mes internos vs 500M externos
- Identificación del problema de hair-pinning y su impacto en latencia (25-50ms por salto)
- Análisis de bottlenecks de performance en arquitectura actual
- Evaluación de overhead de API Management tradicional vs necesidades reales

Insight crítico descubierto:

- **80% del tráfico es interno (service-to-service)**
- Requiere ruteo de alto rendimiento, no overhead de gestión
- Priorizar funcionalidades de **API Gateway** por sobre capacidades clásicas de APIM
- Separación necesaria: Internal Gateway (East-West) vs External API Manager (North-South)
- Fixed pricing es requisito no negociable para tráfico interno

Vendors evaluados en esta fase:

- **Kong Enterprise:**
  - Arquitectura sólida y moderna, Kubernetes-native
  - Amplio ecosistema de plugins (90+)
  - Referencias bancarias (Bradesco)
  - POC completa desplegada
  - Configuración declarativa vía CRDs
- **Traefik Hub:**
  - Costo-efectivo con fixed instance-based pricing
  - Gateway API native support
  - Arquitectura moderna y cloud-native
  - Multi-cluster management console
- **Tyk Enterprise:**
  - Fixed pricing model atractivo
  - Experiencia en open banking
  - Soporte multiclúster nativo
  - Operador de Kubernetes
  - Control Plane Híbrido para multiclusters/multiregion

Actividades realizadas:

- POC completa de Kong Enterprise en cluster de pruebas
- Demos técnicos de Traefik Hub y Tyk
- Pruebas de performance comparativas
- Análisis detallado de modelos de pricing
- Evaluación de integración con OpenShift
- Modelado de solución arquitectónica con integración de networking

Problemas identificados:

- **Kong:**
  - Costos variables East-West muy altos (pricing problemático para 7.5B requests/mes)
  - Limitaciones técnicas en Kong Ingress Controller (KIC)
  - Deprecación de nginx como ingress
  - Arquitectura Hybrid consolida cambios en archivo de configuración de gran tamaño (limita escalabilidad)
- **Traefik:**
  - Soporte LATAM limitado
  - Capacidades Enterprise pobres (poca experiencia con clientes enterprise)
  - Observabilidad pobre (requiere desarrollo de tableros propios de Grafana)

    - Algunas inconsistencias de configuración durante pruebas de estrés
- **Tyk:**
    - Curva de aprendizaje significativa
    - Poca madurez comprobada en implementaciones a gran escala en la región
    - Sin soporte para Gateway API (en desarrollo)
    - Documentación limitada para despliegues complejos
    - Soporte en español y presencia local aún en desarrollo

Solución Modelada: "Galicia Mesh"

Se modeló una solución arquitectónica que requería **fuerte integración de networking** (F5, Infoblox, AppViewX) con automatización bidireccional, sincronización de configuraciones, y failover coordinado.

Implicaciones identificadas:

- **Inversión significativa en equipamiento** (Infoblox, F5) no contemplada en presupuesto
- **Transformación cultural y organizacional** requerida: automatización completa, colaboración inter-equipos (Networking, Seguridad, Platform, DevOps), cambio de procesos a DevOps e "infrastructure as code"
- **Complejidad de implementación:** integraciones custom, pipelines complejos, testing exhaustivo

Conclusión: Si bien ofrecía capacidades avanzadas, la inversión, transformación cultural y complejidad llevaron a buscar alternativas que redujeran dependencia de transformaciones profundas en networking.

Insight crítico descubierto:

El enfoque de API Gateway puro es válido, pero se requiere una solución que combine eficiencia a través de la automatización y capacidades de gestión proactiva para el tráfico multiclúster, manteniendo un modelo de pricing sostenible. El modelado de "Galicia Mesh" reveló que la automatización compleja de networking entre múltiples sistemas es un desafío significativo que podría resolverse con tecnologías nativas de service mesh.

2.3 Fase 3: Priorización de Enfoque Multiclúster y Nueva Oportunidad para Service Mesh

Período: Noviembre 2025 - Enero 2026

Objetivo: Evaluar soluciones de service mesh sidecarless para tráfico East-West multiclúster y observabilidad eBPF.

Contexto:
Los aprendizajes del modelado de Fase 2 ("Galicia Mesh") llevaron a priorizar el **enfoque multiclúster** como requisito crítico, identificando que service discovery y automatización de networking son funcionalidades nativas de Service Mesh moderno sidecarless (en ese momento, con hipótesis inicial sobre ambient mesh).

Actividades realizadas:

- Evaluación de Cilium Cluster Mesh para networking multiclúster L3/L4
- Evaluación de Gloo Gateway para necesidades L7 (en paralelo)
- Análisis de observabilidad eBPF como requisito previo a multiclúster

Alternativas evaluadas:

1. Cilium Cluster Mesh:

- Service Mesh sin sidecars con CNI propia, conectividad multiclúster nativa
- Certificado por Red Hat, observabilidad eBPF avanzada
- **Hallazgos:**
- ✅ Excelente para conectividad multiclúster y networking L3/L4
- ✅ Service discovery nativo entre clusters
- ✅ Observabilidad avanzada basada en eBPF
- ✅ Alto rendimiento y bajo overhead
- ❌ **Limitación crítica:** No resuelve necesidades de L7 (API Gateway, rate limiting, transformación, etc.)
- ❌ **Gap identificado:** Requiere complemento para capacidades de API Management

2. Solo.io Gloo Gateway:

- Basado en Envoy, Gateway API nativo, fixed pricing negociable
- Evaluado en paralelo porque Cilium no resuelve L7
- **Hallazgos:**

- ✅ Arquitectura sólida basada en Envoy
- ✅ Fixed pricing negociable (crítico para tráfico interno)
- ✅ Soporte nativo para Gateway API
- ✅ Base open source (Envoy) reduce vendor lock-in
- ✅ Resuelve necesidades de L7 (API Gateway, rate limiting, transformación)
- ⚠️ Solo resuelve tráfico North-South
- ⚠️ **Gap identificado:** Necesita complemento para tráfico East-West multiclúster

Descubrimiento Crítico de la Fase: Necesidad de eBPF antes de Multiclúster

Durante la evaluación de Cilium Cluster Mesh, se descubrió que la **observabilidad basada en eBPF es un requisito clave antes de implementar arquitectura multiclúster**.

Problema identificado:

- **Deuda técnica crítica:** Falta de mapa de servicios completo y actualizado
- **Observabilidad rota:** 3scale actúa como "firewall" cortando traces completos
- **Falta de visibilidad end-to-end:** Imposible rastrear requests a través de múltiples clusters
- **Sin métricas avanzadas:** Falta de métricas de rendimiento y trazabilidad end-to-end

Ventajas clave de Observabilidad eBPF para Multiclúster:

1. Observabilidad End-to-End (E2E):

- **Visibilidad completa:** Traces completos a través de múltiples clusters sin gaps
- **Sin instrumentación de aplicaciones:** Observabilidad transparente sin modificar código
- **Captura a nivel de kernel:** Métricas y traces capturados en el kernel, no en la aplicación
- **Performance sin overhead:** Mínimo impacto en rendimiento de aplicaciones

2. Mapa de Servicios (Deuda Técnica Crítica):

- **Service discovery visual:** Mapa completo de servicios y dependencias entre clusters
- **Detección automática:** Descubrimiento automático de servicios sin configuración manual
- **Dependencias identificadas:** Visualización clara de relaciones entre servicios
- **Actualización en tiempo real:** Mapa siempre actualizado sin intervención manual
- **Ejemplo:** Herramientas como coroot demuestran el poder de observabilidad eBPF

3. Capacidades Técnicas:

- **Observabilidad a nivel de kernel:** Captura de métricas sin overhead de aplicación
- **Trazabilidad completa:** End-to-end tracing sin gaps causados por 3scale
- **Métricas avanzadas:** Latencia, throughput, errores a nivel de servicio y cluster
- **Network policies observables:** Visibilidad de políticas de red aplicadas

4. Beneficios para Implementación Multiclúster:

- **Validación de conectividad:** Verificar que servicios entre clusters se descubren correctamente
- **Debugging simplificado:** Identificar problemas de conectividad multiclúster rápidamente
- **Optimización de tráfico:** Identificar patrones de tráfico y optimizar rutas entre clusters
- **Monitoreo de salud:** Health checks y métricas de salud a nivel multiclúster

Insight crítico descubierto:

1. **Hipótesis de Fase:** Service discovery y automatización de networking entre clusters son funcionalidades nativas de ambient mesh, eliminando la necesidad de integraciones custom complejas requeridas en "Galicia Mesh" y reduciendo overhead operativo vs sidecars tradicionales. *(Posteriormente descartado para East-West en Fase 4 por issue reproducible.)*
2. **eBPF es Requisito Previo Crítico:** La observabilidad basada en eBPF es esencial antes de implementar arquitectura multiclúster. Resuelve la deuda técnica crítica del mapa de servicios y habilita observabilidad end-to-end necesaria para operar múltiples clusters de forma efectiva.
3. **Gap de Solución Completa (contexto de fase):**
   - Cilium Cluster Mesh resuelve networking multiclúster L3/L4 pero no L7
   - Gloo Gateway resuelve necesidades L7 pero requiere complemento para East-West multiclúster
   - **Necesidad identificada en ese momento:** Evaluar si Gloo Mesh (service mesh ambient) puede complementar Gloo Gateway para solución completa multiclúster

2.4 Fase 4: PoCs de Ambient Mesh y Resolución con Vendor (Cerrada)

Período: Enero - Febrero 2026

Objetivo: Validar estabilidad multiclúster real para tráfico East-West con service mesh sidecarless y cerrar decisión de vendor.

Contexto:
Luego de la Fase 3 se priorizó validar una combinación completa para East-West + North-South. Se ejecutaron PoCs específicas con Solo.io/Gloo Mesh (basado en Istio Ambient) para confirmar comportamiento operativo ante cambios de endpoints en servicios globales.

Actividades realizadas con vendor y equipos internos:

- PoC multiclúster en OpenShift on-prem (OCP 4.18.21) con 2 clusters conectados por East-West Gateways
- Despliegue de Gloo Mesh / Istio Ambient con Gateway API (`Gateway` + `HTTPRoute`) y servicios globales (`ServiceExport` / `ServiceImport`)
- Pruebas de resiliencia sobre tráfico `cluster1 -> cluster2` con reciclado de pods backend
- Validación técnica de causa raíz en nodos usando `oc debug node` , `crictl` y `nsenter` sobre `ztunnel`
- Escalación y seguimiento con Solo.io sobre evidencia de conexiones TCP persistentes a endpoints viejos
- Segunda PoC en OpenShift sobre EC2 para descartar sesgo de infraestructura on-prem
- Ejecución de escenarios sugeridos por vendor (incluyendo failover del workshop público) para contraste de comportamiento

Resultado técnico reproducible

- En tráfico cross-cluster, al reciclar pod backend en `cluster2` (cambio de IP), las requests HTTP desde `cluster1` quedan colgadas indefinidamente.
- La causa validada fue conexión TCP `ESTABLISHED` stale en `ztunnel` hacia la IP antigua del pod remoto (`:15008` HBONE), reutilizada tras cambio de endpoint.
- Reiniciar los pods East-West (`ztunnel`) restablece el tráfico, lo que confirma workaround pero no solución de fondo.
- El problema no se reproduce en tráfico same-cluster.

Validación cruzada de entorno y guía de vendor

- La misma falla se reprodujo en on-prem y en OCP sobre AWS/EC2 con configuración por defecto.
- Los tests sugeridos por vendor y el escenario de failover del workshop no reprodujeron el problema porque no ejercen el mismo patrón de tráfico con reciclado de IP en servicio global remoto.
- Conclusión técnica: el comportamiento observado no es específico del entorno, sino del patrón `ingreso por cluster1 + rotación de IP de pod en cluster2` .

Resolución de la fase

- **Istio Ambient Mesh (incluyendo Gloo Mesh para East-West) descartado** para esta iniciativa por riesgo operativo en patrón crítico y reproducible.
- **El modelo arquitectónico no cambia** (3 capas North-South y malla East-West multiclúster activo-activo).
- **Cambio de componente:** East-West pasa a **Cilium Mesh (Isovalent Enterprise)**, manteniendo API Gateway para North-South.

Estado actual: Decisión de arquitectura cerrada para East-West; en ejecución el plan de implementación sobre Cilium Mesh Enterprise.

3. Lecciones Aprendidas

3.1. Redefinición de la Comunicación entre Servicios

Visión tradicional:

- APIs como interfaces externas con alto overhead de gestión
- Enfoque en capacidades de API Management (developer portal, analytics, monetización)

Realidad descubierta:

- **80% de comunicación interna** que requiere networking de alto rendimiento
- El overhead de API Management tradicional es contraproducente para tráfico interno
- Necesidad de separación clara: Internal Gateway vs External API Manager

Impacto de negocio:

- Ahorros anuales superiores a USD 1M mediante optimización del tráfico interno

- Reducción de latencia de 25-50ms eliminando hair-pinning
- Habilitación de comunicación directa pod-a-pod

3.2. Evolución del Modelo de Seguridad

Enfoque legado:

- Credenciales estáticas (API keys sin expiración)
- Fricción operativa alta (200+ horas mensuales de ingeniería)
- Hallazgos de auditoría regulatoria

Estado objetivo:

- mTLS entre servicios
- Integración dinámica JWT/OAuth con sistemas corporativos de identidad
- Gestión automatizada del ciclo de vida de credenciales
- Token expiration y revocación automática

Mitigación de riesgo:

- Reducción del 70% en incidentes de seguridad mediante automatización
- Cumplimiento regulatorio mejorado
- Eliminación de anti-patrones de seguridad

3. Transformación de la Arquitectura de Alta Disponibilidad

Limitación actual:

- DR activo-pasivo con failover manual
- Sincronización manual vía pipelines
- RTO superior a 4 horas
- Riesgo de desincronización entre sitios

Estado futuro:

- Multiclúster activo-activo con ruteo automatizado basado en salud
- Sincronización declarativa (Infrastructure as Code)
- Failover automático sub-segundo
- Control plane centralizado con data planes autónomos

Protección de ingresos:

- Mitigación de riesgos por más de USD 20M anuales mediante failover automático
- Eliminación de errores manuales en procesos de DR
- Reducción de RTO de 4 horas a <1 segundo

4. Separación de Control Plane y Data Plane

Lección crítica:

- Arquitectura desacoplada es esencial para escalabilidad y resiliencia
- Data planes deben funcionar independientemente si pierden conectividad con control plane
- Permite escalar y actualizar de forma independiente

Aplicación práctica:

- Control plane puede estar en nube pública (SaaS) o gestionado pero desacoplados (sin SPOF en común)
- Data planes distribuidos en múltiples clusters/sitios
- Modelo hub-and-spoke para gestión centralizada

5. Importancia del Fixed Pricing

Problema identificado:

- Modelos de pricing por API call son inviables para 7.5B requests/mes internos
- Costos impredecibles con crecimiento orgánico
- Burst traffic puede generar costos explosivos

Solución requerida:

- Fixed pricing o costos predecibles
- Sin penalización por burst o crecimiento orgánico
- Crítico para sostenibilidad financiera a largo plazo

6. Declarative Configuration es No Negociable

Problema actual:

- 3scale: Todo en base de datos, sin capacidad declarativa
- Migración manual de 2,200 APIs sin herramientas
- Imposible GitOps workflows

Requerimiento futuro:

- Todo debe ser declarativo (CRDs, YAML)
- GitOps workflows nativos
- Versionado y auditoría de cambios
- Sincronización automática entre sitios

7. Kubernetes Gateway API como Estándar

Insight:

- Gateway API es el futuro de Kubernetes networking
- Evita dependencias en implementaciones propietarias
- Permite interoperabilidad y portabilidad
- Facilita separación de roles (admin de infra vs aprovisionadores de rutas)

Requerimiento:

- Soporte nativo para K8s Gateway API es must-have
- Facilita integración con ecosistemas cloud native
- Posibilita definiciones declarativas multicluster

8. Ambient Mesh vs Cilium Mesh (Validación en Tráfico Real)

Evaluación:

- Sidecar mesh: Overhead operativo alto, complejidad de gestión
- Sidecarless mesh: Sigue siendo el enfoque correcto para escala y operación
- Istio ambient mesh: Falla reproducible en tráfico cross-cluster cuando se recicla la IP del pod backend remoto
- Los escenarios de failover sugeridos por vendor no reprodujeron esta falla específica porque ejercen un patrón distinto de tráfico

Decisión:

- Mantener la decisión de arquitectura **sidecarless**
- Descartar Istio ambient mesh para el caso de uso East-West multiclúster de esta iniciativa
- Adoptar **Cilium Mesh (Isovalent Enterprise)** para East-West, preservando el modelo de arquitectura definido
- Incorporar pruebas obligatorias de "pod churn cross-cluster" como criterio de aceptación en POC y preproducción

4. Decisiones Técnicas y Arquitectónicas

Decisión 1: Service Mesh Sidecarless + API Gateway (Stack Definido)

Decisión: Implementar **Service Mesh Sidecarless (Cilium Mesh Enterprise) + API Gateway** como solución principal.

Justificación técnica:

- **Service mesh sidecarless:** Elimina overhead operativo y complejidad de gestión vs sidecars tradicionales
- **Service mesh multiclúster nativo:** Elimina hair-pinning mediante comunicación directa pod-a-pod
- **Arquitectura cloud-native abierta:** Basada en estándares Kubernetes (Gateway API), eBPF y componentes open source
- **Modelo de pricing fijo:** Evita explosión de costos por volumen de tráfico interno

- **Riesgo técnico validado en PoC:** Istio Ambient Mesh fue descartado por falla reproducible de conexiones stale cross-cluster tras reciclado de pods
- **Base open source:** Cilium/eBPF + Gateway API minimizan lock-in

Stack seleccionado:

- **Service Mesh East-West:** Cilium Mesh (Isovalent Enterprise)
- **API Gateway North-South:** Gloo Gateway (preferencia actual)

Componentes:

- **Service Mesh Sidecarless:** Para tráfico East-West (Cilium Mesh Enterprise)
- **API Gateway:** Para tráfico North-South y gestión de APIs externas (preferencia por Gloo Gateway).

Decisión 2: Despliegue On-Demand de capacidades avanzadas de L7

Decisión: Desplegar **API Gateway on-demand en namespaces que lo requieran**.

Justificación:

- No todos los namespaces requieren capacidades de API Gateway
- Reducción de overhead operativo y de recursos
- Modelo de despliegue granular y eficiente
- Permite migración gradual namespace por namespace
- Facilita testing y validación incremental
- Facil instrumentacion
- Gestionado por DevOps teams

Implementación:

- Gateway desplegado como operador de Kubernetes en namespaces que requieran capacidades L7 avanzadas
- Configuración declarativa por namespace
- Integración con GitOps workflows
- Observabilidad auto instrumentada

Decisión 3: Arquitectura de 3 Capas para Tráfico North-South

Decisión: Implementar arquitectura de **3 capas para tráfico North-South: DMZ → API GW (B2B) → Mesh**.

Capa 1: DMZ (Demilitarized Zone)

- **Función:** Punto de entrada externo, protección perimetral
- **Componentes:** F5 load balancers, firewalls, DDoS protection, Proxy Reversos
- **Responsabilidades:**
  - Terminación SSL/TLS inicial
  - Autenticacion contra APIs de 3eros (Partners)
  - Filtrado de tráfico malicioso
  - Rate limiting básico
  - Enrutamiento inicial hacia API Gateway

> **IMPORTANTE**: El proveniente de sistemas Legacy, Core Bancario o cualquier otro componente que consuma APIs alojadas en Openshift se considerar trafico Norte/Sur

Capa 2: API Gateway (B2B / B2C)

- **Función:** Gestión de APIs externas a Openshift, autenticación B2B, políticas de negocio
- **Componente:** API Gateway desplegado en namespace dedicado o cluster dedicado (En Analisis)
- **Responsabilidades:**
  - Autenticación y autorización (OAuth2, JWT, mTLS)
  - Rate limiting y throttling por cliente/API
  - Transformación de requests/responses
  - Analytics y observabilidad de APIs externas
  - Developer portal para partners externos
  - Gestión de versionado de APIs
  - Políticas de negocio (quota management, monetización)

Capa 3: Service Mesh

- **Función:** Comunicación interna, service-to-service, tráfico East-West
- **Componente:** Cilium Mesh (Isovalent Enterprise)
- **Responsabilidades:**
  - Service discovery y routing interno via E/W Gateways
  - mTLS entre servicios
  - Observabilidad end-to-end (tracing, metrics, logs de infra)
  - Políticas de seguridad L4
  - Políticas L7 on-demand mediante componentes dedicados donde aplique
  - Circuit breakers y retry policies
  - Canary deployments y traffic splitting

Beneficios:

- **Separación de responsabilidades:** Cada capa tiene un propósito claro
- **Seguridad en profundidad:** Múltiples capas de protección
- **Escalabilidad independiente:** Cada capa escala según necesidad
- **Observabilidad granular:** Métricas y traces por capa
- **Flexibilidad:** Cambios en una capa no afectan otras

Decisión 4: Multiclúster Activo-Activo

Decisión: Implementar arquitectura **multiclúster activo-activo** con failover automático.

Justificación:

- Eliminación de RTO manual (4 horas → <1 segundo)
- Distribución de carga entre sitios
- Resiliencia ante fallos de sitio completo
- Sincronización automática de configuraciones

Implementación:

- Control plane centralizado (hub-and-spoke)
- Data planes autónomos en cada cluster
- Sincronización declarativa vía GitOps
- Failover automático basado en health checks
- DNS automático para routing

Decisión 5: Separación de Responsabilidades L4/L7 en Service Mesh

Decisión: La plataforma provee capacidades completas de capa 4 (L4) de forma centralizada, mientras que las capacidades avanzadas de capa 7 (L7) serán instrumentadas por DevOps según necesidad específica.

Justificación:

- Reduce complejidad operativa eliminando la implementacion dentro de la aplicación
- Permite adopción gradual de funcionalidades avanzadas
- Minimiza overhead de infraestructura
- Delega control granular a equipos que mejor conocen sus aplicaciones
- Mantiene rendimiento óptimo para servicios que no requieren L7

Implementación:

- **Plataforma (L4):** Agente por nodo manejando conectibidad TCP/UDP, mTLS automático, service discovery global, comunicación inter-cluster
- **DevOps (L7):** Proxys por namespace para HTTP routing, autenticación específica, rate limiting, canary deployment, circuit breakers
- **Control:** Plataforma gestiona conectividad base, DevOps instrumenta políticas de aplicación
- **Escalabilidad:** Modelo "a pedido" - solo despliega L7 donde es necesario

Decisión 6: Gobierno de APIs mediante Políticas Nativas del Service Mesh

Decisión: Mantener el gobierno de APIs (control de quién consume qué) utilizando authentication policies nativas del service mesh, eliminando network policies restrictivas y aprovechando la observabilidad completa integrada.

Justificación:

- Eliminación del hair-pinning actual (evitar modificar el flujo/trafico para autenticar comunicaciones internas entre servicios)
- Observabilidad end-to-end nativa vs trazas cortadas por API gateway
- Políticas declarativas replicadas automáticamente entre clusters
- Autenticación a nivel de identidades de k8s (ej: service account) vs API keys estáticas
- Reducción de latencia eliminando saltos innecesarios por load balancer

Implementación:

- **Authentication Policies:** Basadas en service accounts por namespace, eliminando API keys estáticas
- **Suscripción declarativa por CRD:** Trazabilidad por git y de fácil integración con actual proceso de suscripción
- **Observabilidad Nativa:** Métricas L4/L7, mapeo de servicios con eBPF, análisis estacional de tráfico
- **Políticas Globales:** Control planes federados distribuyendo authorization policies a todos los clusters
- **Service Discovery:** Comunicación directa intra-cluster eliminando dependencia del DNS corporativo
- **Auditoría:** Trazabilidad completa de suscripciones y accesos por namespace

Decisión 7: Fixed Pricing como Requisito

Decisión: Fixed pricing es requisito no negociable para tráfico interno.

Justificación:

- 7.5B requests/mes hacen inviable pricing por API call
- Costos predecibles para planificación financiera
- Sin penalización por crecimiento orgánico
- Sostenibilidad a largo plazo

Negociación:

- Fixed pricing para tráfico East-West (interno)
- Pricing negociable para tráfico North-South (externo)
- Modelo híbrido aceptable si se mantiene predictibilidad

Decisión 8: Kubernetes Gateway API como Estándar

Decisión: Soporte nativo para Kubernetes Gateway API es must-have.

Justificación:

- Estándar emergente de la industria
- Evita vendor lock-in
- Facilita portabilidad entre entornos
- Permite separación de roles (infra vs developers)

Implementación:

- Todas las rutas definidas usando Gateway API CRDs
- Compatibilidad con estándar Gateway API v1.2+
- Integración con ecosistemas cloud native

Decisión 9: Declarative Configuration (GitOps)

Decisión: Toda la configuración debe ser **declarativa y gestionada vía GitOps**.

Justificación:

- Versionado y auditoría de cambios
- Sincronización automática entre sitios
- Rollback automatizado
- Eliminación de drift de configuración

Implementación:

- CRDs para todas las configuraciones
- GitOps workflows (ArgoCD)
- CI/CD pipelines para validación

- Automated testing de configuraciones

Decisión 10: Descarte de Istio Ambient para East-West y Adopción de Cilium Mesh

Decisión: Descartar **Istio Ambient Mesh** para tráfico East-West multiclúster de esta iniciativa y adoptar **Cilium Mesh (Isovalent Enterprise)**.

Justificación:

- PoCs reproducibles mostraron cuelgue de tráfico cross-cluster tras reciclado de pod backend remoto
- Causa raíz validada: conexión TCP stale en `ztunnel` hacia IP vieja del endpoint (`:15008` HBONE)
- Reproducido en on-prem y en OpenShift sobre EC2, descartando sesgo de infraestructura
- Workaround existente (reinicio de East-West gateway) no es aceptable como estrategia operativa
- El riesgo de disponibilidad para patrón crítico de negocio es alto

Implementación:

- Mantener arquitectura sidecarless y modelo multiclúster activo-activo
- Reemplazar componente East-West por Cilium Mesh Enterprise
- Mantener API Gateway para North-South y capacidades L7
- Incluir pruebas de "pod churn cross-cluster" como criterio obligatorio de pase a producción

5. Arquitectura Objetivo

5.1 Resumen Ejecutivo

Transformación desde cluster monolítico con 3scale hacia **arquitectura multiclúster activo-activo** con separación clara de responsabilidades:

Componentes Principales:

- **Service Mesh Sidecarless (Cilium Mesh Enterprise):** Tráfico interno East-West (7.5B requests/mes), comunicación directa pod-a-pod eliminando hair-pinning
- **API Gateway:** Tráfico externo North-South de Openshift, arquitectura de 3 capas (DMZ/Core/Legacy → API Gateway → Mesh)
- **Separación L4/L7:** Plataforma gestiona conectividad base (L4), DevOps instrumenta políticas avanzadas (L7) on-demand
- **Gobierno nativo:** Políticas del service mesh basadas en identidades Kubernetes vs API keys estáticas

Beneficios Clave:

- Eliminación de hair-pinning (reducción notable de latencia)
- Observabilidad end-to-end con eBPF vs trazas cortadas dificil de armar el e2e
- Failover automático sub-segundo vs 4 horas manual
- Configuración declarativa GitOps vs procesos manuales
- Autenticación moderna (mTLS, OAuth2/JWT) vs API keys estáticas

Estado: Arquitectura definida y stack East-West cerrado en Cilium Mesh Enterprise tras PoCs multiclúster.

5.2 Visión General

```
┌──────────────────────────────────────────────────────────────────────┐
│                        TRÁFICO NORTH-SOUTH                           │
│                                                                      │
│  Internet                                                            │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────┐                      ┌─────────┐                          │
│  │ DMZ │  (F5, Firewalls, │ Other  │  (Core Banking,              │
│  └──────┘  DDoS Protection)└─────────┘  Sistemas Legados)           │
│    │                          │                                      │
│    ▼                          ▼                                      │
│  ┌──────────────────────────────────┐                                │
│  │ API Gateway (B2B/B2C)            │  (API GW Layer -               │
│  │       - OAuth2/JWT               │      cluster dedicado)         │
│  │       - Rate Limiting            │                                │
│  │       - API Versioning           │                                │
│  │       - Developer Portal         │                                │
│  └──────────────────────────────────┘                                │
│    │                          ▼                                      │
│    ▼                      Backend Services (NO OCP)                  │
│  ┌──────────────────────┐                                            │
│  │   Service Mesh       │  (Sidecarless/Cilium Mesh)                 │
│  │   - mTLS             │                                            │
│  │   - Observability    │                                            │
│  │   - L7 Policies      │                                            │
│  └──────────────────────┘                                            │
│    │                                                                 │
│    ▼                                                                 │
│  Backend Services (Microservices on OCP)                             │
└──────────────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────────┐
│                        TRÁFICO EAST-WEST                             │
│                                                                      │
│  Service A (Namespace 1)                                             │
│    │                                                                 │
│    ▼                                                                 │
│  ┌──────────────────────┐                                            │
│  │   Service Mesh       │  (Sidecarless - Direct Pod-to-Pod)         │
│  │   - No Hair-pinning  │                                            │
│  │   - mTLS             │                                            │
│  │   - Service Discovery│                                            │
│  └──────────────────────┘                                            │
│    │                                                                 │
│    ▼                                                                 │
│  Service B (Namespace 2)                                             │
└──────────────────────────────────────────────────────────────────────┘
```

5.3 Componentes Clave

A. Control Plane Centralizado

API Gateway Management Plane:

- Gestión centralizada de múltiples clusters
- Sincronización de políticas y configuraciones
- Observabilidad agregada
- Puede estar en nube pública (SaaS) o on-premise

Características:

- Hub-and-spoke model
- Data planes autónomos (funcionan sin control plane)
- Sincronización declarativa
- Multi-tenancy support

B. Data Planes Distribuidos

Por Cluster/Sitio:

- Service Mesh Sidecarless data plane (Cilium Mesh Enterprise)
- Componentes L7 on-demand por namespace cuando aplique
- East-West Gateway (east-west gw)

Características:

- Autónomos (funcionan sin control plane)
- Sincronización automática desde unica fuente de verdad
- Health checks y failover automático
- Observabilidad local y centralizada

C. Despliegue On-Demand de API Gateways

Modelo:

- Gateway desplegado solo en namespaces que lo requieren
- Configuración declarativa por namespace
- Integración con GitOps
- Facil instrumentacion
- Integración con la Mesh

Namespaces típicos que requieren Gateway:

- APIs expuestas hacia afuera de Openshift que requieran alguna autenticacion adicional, rate limits, manejo avanzado de header, etc
- Partner integrations
- Public APIs
- Terminacion mTLS

D. Integración con Infraestructura Existente

F5 Load Balancers:

- Terminación SSL/TLS inicial
- Enrutamiento hacia API Gateway
- Health checks

OpenShift:

- Certificación y compatibilidad nativa
- Integración con operadores de Kubernetes
- Network policies
- Service accounts y RBAC

DNS:

- Automatización de updates DNS corporativo
- Failover automático de DNS
- Multi-site routing

Separación de Tráfico

5.4 Tráfico North-South (Externo)

Flujo:

1. Internet → DMZ (F5/ Core / Legacy)
2. DMZ → API Gateway (Gloo Gateway/ Otro)
3. API Gateway → Service Mesh (Sidecarless)
4. Service Mesh → Backend Services

Características:

- Autenticación OAuth2/JWT
- Rate limiting por cliente/API

- Analytics y observabilidad
- Developer portal
- API versioning

Tráfico East-West (Interno)

Flujo:

1. Service A → Service Mesh (Cilium Mesh)
2. Service Mesh → Service B (direct pod-to-pod)

Características:

- Sin hair-pinning
- mTLS automático
- Service discovery
- Observabilidad end-to-end
- L7 policies (opcional, con componentes dedicados por namespace)

Alta Disponibilidad y Disaster Recovery

Arquitectura Multiclúster:

- **Sitio 1 (Plaza):** Cluster activo
- **Sitio 2 (Matriz):** Cluster activo
- **Modo:** Activo-activo con distribución de carga

Failover Automático:

- Health checks continuos
- Failover automático sub-segundo
- DNS automático
- Sin intervención manual

Sincronización:

- Configuraciones declarativas (GitOps)
- Sincronización automática entre sitios
- Sin riesgo de drift
- Rollback automatizado

6. Evaluación de Proveedores

Criterios de Evaluación

Must-Haves Definitivos:

1. **Soporte multiclúster**
   - Despliegue y administración en múltiples clusters K8s
   - Alta disponibilidad (HA) entre regiones y datacenters
   - Failover y recuperación ante desastres entre clusters
   - Replicación y sincronización de configuraciones y políticas
   - Visibilidad y monitoreo centralizado de gateways multiclúster
2. **Sin vendor Lock-in en OpenShift**
   - Compatibilidad certificada con OpenShift Container Platform
   - Sin dependencias propietarias que generen lock-in con el proveedor o cloud específica
   - Portabilidad entre entornos
3. **Fixed pricing**
   - Modelo de costos predecible y fijo
   - Sin penalización por burst o crecimiento orgánico
   - Crítico para 7.5B requests/mes en tráfico interno (East-West)
4. **Envoy Proxy Gateway**
   - Envoy como data plane del gateway: tecnología robusta y probada

- Estándar del mercado, respaldo de una comunidad activa e innovación constante
- Garantiza escalabilidad, observabilidad avanzada y máxima flexibilidad/integración

5. **Soporte nativo para K8s Gateway API**
   - Compatibilidad total con el estándar Gateway API de Kubernetes
   - Permite integración directa con ecosistemas cloud native
   - Facilita definiciones declarativas, separación de roles y mayor flexibilidad en escenarios multicluster
6. **Desacople de Control Plane y Data Plane**
   - Arquitectura separada entre Control Plane y Data Plane
   - Capacidad de escalar y actualizar de forma independiente
   - Data planes autónomos que funcionan independientemente si pierden conectividad con control plane
   - Depliegue Hybrid - Control Plane gestionado como SaaS o en nube pública
7. **Backends externos a Kubernetes**
   - Exposición y gestión de servicios backends ubicados fuera del clúster de Kubernetes
   - Soporte para integración con backends en redes internas o públicas, functions (ie. AWS Lambda), máquinas virtuales o servicios legacy externos

Nice-to-Have:

- IA Gateway Ready (capacidades para gobernanza de APIs de IA/ML)
- Developer portal avanzado
- Analytics y monetización

Evaluación Detallada de Vendors

Solo.io (Gloo Mesh + Gloo Gateway) - EVALUADO Y DESCARTADO PARA EAST-WEST

Fortalezas:

- ✅ Arquitectura basada en Envoy (estándar de la industria)
- ✅ Service mesh multiclúster nativo
- ✅ Ambient mesh (reduce complejidad vs sidecars)
- ✅ Fixed pricing negociable
- ✅ Top contributor a Istio/Envoy
- ✅ Base open source (Istio/Envoy)
- ✅ Soporte para Gateway API
- ✅ Control plane híbrido (SaaS o on-premise)
- ✅ Backends externos a Kubernetes
- ✅ Fuerte en financial services

Limitaciones:

- ❌ Issue crítico reproducible en PoC de Ambient Mesh: tráfico cross-cluster queda colgado tras reciclado de pod backend remoto
- ❌ Causa validada: conexión TCP stale en `ztunnel` hacia IP vieja del endpoint (`:15008` HBONE)
- ❌ Reproducido en OpenShift on-prem y OpenShift sobre EC2 (no dependiente del entorno)
- ⚠️ Workaround disponible (reinicio de East-West gateway), no aceptable como solución operativa
- ⚠️ Producto más nuevo (menor market presence que Kong)
- ⚠️ Soporte principalmente en inglés (US/Europa)
- ⚠️ Portal funcionalidad inmadura (releasing Marzo 2025)
- ⚠️ Curva de aprendizaje

Estado: Descartado para tráfico East-West de esta iniciativa. Gloo Gateway puede mantenerse como opción para North-South.

Kong Enterprise

Fortalezas:

- ✅ Producto maduro con amplia comunidad
- ✅ Arquitectura sólida y moderna
- ✅ Integración nativa con Kubernetes
- ✅ Amplio ecosistema de plugins (90+)
- ✅ Referencias bancarias (Bradesco)
- ✅ Configuración declarativa vía CRDs
- ✅ Estrategia side-by-side para migración

Limitaciones:

- ❌ Costos variables East-West muy altos (pricing problemático)
- ❌ Limitaciones técnicas en KIC (deprecación de nginx como ingress)
- ❌ Arquitectura Hybrid consolida cambios en archivo de configuración de gran tamaño (limita escalabilidad)
- ❌ Sin built-in multi-cluster federation
- ⚠️ Soporte limitado en español
- ⚠️ Overhead de infraestructura significativo

Estado: POC completa, descartado por pricing

Red Hat Connectivity Link

Fortalezas:

- ✅ Integración nativa OpenShift
- ✅ HA nativo
- ✅ Envoy API GW
- ✅ Operador de K8s
- ✅ Continuidad con Red Hat
- ✅ Migración asistida desde 3scale
- ✅ 50% descuento primer año para migraciones 3scale
- ✅ Automatic DNS failover y HA

Limitaciones:

- ❌ Producto muy nuevo (v1.0, limitadas implementaciones en producción)
- ❌ No resuelve API Management completo
- ❌ Inadecuado para tráfico norte-sur
- ❌ Precio por API Call (problemático)
- ❌ Vendor (OpenShift) lock-in
- ❌ No puede mapear APIs externas como servicios internos
- ⚠️ Herramientas de migración automatizada en desarrollo (6-7/10 tasa de éxito actualmente)

Estado: POC on-premises pendiente, considerado como backup

Traefik Hub

Fortalezas:

- ✅ Gateway API native support
- ✅ Costo-efectivo
- ✅ Arquitectura moderna y cloud-native
- ✅ Fixed instance-based pricing
- ✅ Multi-cluster management console
- ✅ Integración con múltiples backends

Limitaciones:

- ❌ Algunas inconsistencias de configuración durante pruebas de estrés
- ❌ Observabilidad pobre (se deben desarrollar propios tableros de Grafana)
- ❌ Soporte LATAM limitado
- ❌ Pobre en capacidades Enterprise (poca experiencia con clientes enterprise)
- ❌ Certificación Red Hat incierta
- ⚠️ Herramientas de migración limitadas

Estado: Pruebas en curso, descartado por limitaciones enterprise

Tyk Enterprise

Fortalezas:

- ✅ Fixed pricing model
- ✅ Experiencia en open banking
- ✅ Soporte multiclúster nativo
- ✅ Operador de K8s
- ✅ Control Plane Híbrido pensado para multiclusters/multiregion

Limitaciones:

- ❌ Sin soporte para Gateway API (en desarrollo)
- ❌ Documentación limitada para despliegues complejos
- ❌ Integraciones con herramientas empresariales aún por validar
- ❌ Curva de aprendizaje
- ❌ Soporte en español y presencia local aún en desarrollo
- ❌ Poca madurez comprobada en implementaciones a gran escala en la región

Estado: Demo Enterprise pendiente, evaluación en curso

Cilium (Isovalent Enterprise) - SELECCIONADO PARA EAST-WEST

Fortalezas:

- ✅ Certificado por Red Hat
- ✅ Super observabilidad (coroot ejemplo)
- ✅ Network Policies L7 (basado en entidades de k8s)
- ✅ Alto rendimiento - Menos saltos
- ✅ Ahora es parte de Cisco
- ✅ Service Mesh sin sidecars y networking basado en eBPF

Limitaciones:

- ⚠️ Evaluación como CNI y como ingress (basado en Envoy) de OpenShift
- ⚠️ No es solución completa de API Management
- ⚠️ Requiere complemento para capacidades de API Gateway North-South

Estado: Seleccionado para tráfico East-West multiclúster. En planificación de implementación enterprise.

Hallazgo Crítico de PoCs Multiclúster con Vendor

- Patrón validado: tráfico ingresa por `cluster1` y consume backend en `cluster2` vía servicio global.
- Al reciclar pod backend en `cluster2` (cambio de IP), requests quedan colgadas con Ambient Mesh.
- Se ejecutaron pruebas sugeridas por vendor, incluyendo escenario de failover del workshop público; no reproducen el problema porque no ejercen el mismo patrón.
- El problema se reprodujo de forma consistente en on-prem y en AWS/EC2.
- Resolución técnica del proyecto: mantener arquitectura sidecarless, reemplazando Ambient Mesh por Cilium Mesh Enterprise para East-West.

Tabla Comparativa

Vendor	Multicluster	DR Automation	Declarative Config	Pricing Model	OpenShift Fit	3scale Migration	Envoy Based	Gateway API
Solo.io/Gloo	⚠️ Validado con issue crítico en EW	⚠️ Afectado en patrón crítico	✅ Strong	Fixed (negociable)	✅ Excellent	Coexistence	✅ Yes	✅ Yes
Kong	⚠️ Limited	⚠️ Manual	✅ Strong	❌ Variable (high)	✅ Good	Side-by-side	✅ Yes	⚠️ Partial
Connectivity Link	✅ Native	✅ Automated	✅ Strong	❌ Per-call (high)	✅ Excellent	✅ Automated tools	✅ Yes	⚠️ Partial
Traefik Hub	✅ Good	⚠️ Manual	✅ Good	✅ Fixed	✅ Good	⚠️ Limited	✅ Yes	✅ Yes
Tyk	✅ Good	⚠️ Manual	✅ Good	✅ Fixed	✅ Good	⚠️ Manual	❌ No	❌ No (dev)
Cilium	✅ Native (L3/L4 multiclúster)	✅ Diseñado para HA multiclúster	✅ Good	✅ Enterprise predecible	✅ Excellent	❌ N/A	⚠️ Parcial (complementar)	⚠️ Parcial (complementar)

7. Pasos Necesarios: Roadmap Técnico

Fase 1: Investigación y Evaluación (Completada)

Estado: Cerrada

Actividades completadas:

- ✅ Evaluación de múltiples vendors (Kong, Tyk, Connectivity Link, Traefik, Apigee)
- ✅ Identificación de must-haves técnicos
- ✅ Análisis de arquitectura actual (3scale)
- ✅ Identificación de pain points y limitaciones
- ✅ Cluster de pruebas desplegado (25/11)
- ✅ PoCs de Ambient Mesh multiclúster en on-prem y AWS/EC2
- ✅ Validación técnica de issue de conexiones stale en `ztunnel` con soporte vendor
- ✅ Decisión de reemplazo de Ambient Mesh por Cilium Mesh Enterprise para East-West

Actividades pendientes:

- ⏳ Cerrar definición contractual/licenciamiento de Cilium Mesh Enterprise (Isovalent)
- ⏳ Diseñar arquitectura detallada de integración Cilium Mesh + API Gateway
- ⏳ Desplegar aplicaciones corporativas (SPA, BFF, backend con persistencia)
- ⏳ Ejecutar PoC de hardening y performance sobre stack seleccionado

Entregables:

- Documento de evaluación de vendors
- Matriz comparativa técnica
- Recomendación de vendor

Fase 2: Rediseño Arquitectónico (Pendiente)

Objetivo: Diseñar arquitectura objetivo detallada basada en decisión de vendor.

Actividades:

1. **Diseño de Arquitectura Detallada**
   - Arquitectura de 3 capas para North-South (DMZ → API GW → Mesh)
   - Arquitectura de Service Mesh para East-West
   - Modelo de despliegue on-demand de Gloo Gateway
   - Integración con infraestructura existente (F5, OpenShift, DNS)
2. **Diseño de Multiclúster**
   - Arquitectura activo-activo entre sitios
   - Sincronización de configuraciones
   - Failover automático
   - DNS automation
3. **Diseño de Seguridad**
   - Modelo de autenticación (OAuth2/JWT)
   - mTLS entre servicios
   - Network policies
   - Integración con sistemas corporativos de identidad
4. **Diseño de Observabilidad**
   - Métricas, logs, traces
   - Dashboards y alertas
   - Integración con herramientas existentes

Entregables:

- Documento de arquitectura detallada
- Diagramas de arquitectura
- Plan de proyecto final

Timeline: 4-6 semanas

Fase 3: Proof of Concept (POC) (Pendiente)

Objetivo: Validar solución seleccionada en ambiente controlado.

Ambiente de POC:

- 2 clusters OpenShift (simulando activo-activo)
- Aplicaciones de prueba (Java, .NET, SPA, BFF)
- Integración con F5, DNS, identity providers

Escenarios de Prueba:

1. **Funcionalidad Básica**
   - Despliegue de Cilium Mesh Enterprise + API Gateway
   - Configuración de rutas básicas
   - Autenticación OAuth2/JWT
   - Rate limiting
2. **Multiclúster**
   - Sincronización de configuraciones entre clusters
   - Failover automático
   - Health checks y routing
3. **Performance**
   - Load testing a escala de producción
   - Latencia y throughput
   - Comparación con 3scale baseline
4. **Integración**
   - Integración con F5
   - Integración con DNS corporativo
   - Integración con identity providers
   - Integración con sistemas de observabilidad
5. **Migración**
   - Coexistencia con 3scale
   - Migración de APIs de prueba
   - Validación de funcionalidad equivalente

Criterios de Éxito:

- ✅ Despliegue exitoso en OpenShift
- ✅ Funcionalidad equivalente a 3scale
- ✅ Performance igual o mejor que 3scale
- ✅ Failover automático funcional
- ✅ Integración con infraestructura existente
- ✅ Migración de APIs sin downtime

Entregables:

- Reporte de POC
- Métricas de performance
- Recomendaciones de implementación
- Plan de migración detallado

Timeline: 8-12 semanas

Fase 4: Pruebas de Performance (Pendiente)

Objetivo: Validar performance bajo carga de producción.

Pruebas Requeridas:

1. **Load Testing**
   - Volumen: 7.5B requests/mes (East-West)
   - Pico: 25,000+ RPS
   - Duración: Tests sostenidos de 24-48 horas
   - Aplicaciones compliance del banco
2. **Métricas a Validar**

    - Latencia (p50, p95, p99)
    - Throughput
    - Error rate
    - Resource utilization (CPU, memoria, red)
    - Comparación con baseline de 3scale
3. **Stress Testing Multiclúster**
    - Failover automático con volumen productivo
    - Comportamiento bajo fallos de cluster
    - Sincronización bajo carga
    - Recovery time
4. **Pruebas de Escalabilidad**
    - Escalado horizontal
    - Comportamiento con crecimiento de tráfico
    - Límites de capacidad

Entregables:

- Reporte de performance
- Métricas comparativas vs 3scale
- Recomendaciones de sizing
- Límites identificados

Timeline: 4-6 semanas (paralelo con POC)

Fase 5: Planificación de Migración (Pendiente)

Objetivo: Desarrollar plan detallado de migración desde 3scale.

Actividades:

1. **Inventario de APIs**
    - Catalogar todas las 2,200 APIs
    - Identificar dependencias
    - Clasificar por criticidad
    - Priorizar orden de migración
2. **Estrategia de Migración**
    - Enfoque gradual (namespace por namespace)
    - Coexistencia con 3scale
    - Ventanas de migración
    - Rollback procedures
3. **Migración de Configuraciones**
    - Mapeo de configuraciones 3scale → API Gateway + Service Mesh
    - Scripts de conversión
    - Validación de equivalencia
    - Testing de configuraciones migradas
4. **Migración de Credenciales**
    - Estrategia de migración de API keys
    - Integración con sistemas de identidad
    - OAuth2/JWT migration
    - Revocación de credenciales antiguas
5. **Plan de Comunicación**
    - Notificación a equipos de desarrollo
    - Documentación para desarrolladores
    - Training y capacitación
    - Support durante migración

Entregables:

- Plan de migración detallado
- Scripts de migración
- Documentación para desarrolladores
- Plan de comunicación

Timeline: 6-8 semanas

Fase 6: Implementación Piloto (Pendiente)

Objetivo: Migrar subset de APIs como prueba piloto.

Selección de APIs Piloto:

- APIs no críticas
- APIs con bajo volumen
- APIs con configuración simple
- Representativas de diferentes casos de uso

Actividades:

1. Despliegue de Cilium Mesh Enterprise + API Gateway en producción
2. Migración de APIs piloto
3. Validación de funcionalidad
4. Monitoreo y observabilidad
5. Ajustes y optimizaciones

Criterios de Éxito:

- ✅ APIs piloto funcionando correctamente
- ✅ Performance igual o mejor que 3scale
- ✅ Sin incidentes críticos
- ✅ Feedback positivo de equipos de desarrollo

Timeline: 8-12 semanas

Fase 7: Migración Gradual (Pendiente)

Objetivo: Migrar todas las APIs de forma gradual.

Estrategia:

- Migración por namespaces
- Migración por equipos de desarrollo
- Migración por criticidad (baja → media → alta)

Actividades:

1. Migración continua de APIs
2. Validación y testing continuo
3. Monitoreo y observabilidad
4. Ajustes y optimizaciones
5. Descommissioning de 3scale

Timeline: 6-12 meses (dependiendo de volumen)

Fase 8: Optimización y Mejora Continua (Pendiente)

Objetivo: Optimizar y mejorar la solución post-migración.

Actividades:

1. Optimización de performance
2. Ajuste de políticas y configuraciones
3. Mejora de observabilidad
4. Training continuo
5. Adopción de nuevas capacidades

Timeline: Continuo

8. Riesgos y Mitigaciones

Riesgos Técnicos

R1: Complejidad de Migración de 2,200 APIs

Riesgo: Migración manual de 2,200 APIs sin herramientas automatizadas puede ser muy compleja y propensa a errores.

Probabilidad: Alta
Impacto: Alto

Mitigación:

- Desarrollo de scripts de migración automatizados
- Migración gradual (namespace por namespace)
- Validación exhaustiva de cada migración
- Coexistencia con 3scale durante período de transición
- Rollback procedures documentados

R2: Performance Inferior a 3scale

Riesgo: La nueva solución puede tener performance inferior a 3scale bajo carga de producción.

Probabilidad: Media
Impacto: Alto

Mitigación:

- Pruebas de performance exhaustivas en POC
- Load testing a escala de producción
- Baseline de performance de 3scale documentado
- Optimización continua post-migración
- Escalado horizontal si necesario

R11: Inestabilidad Cross-Cluster ante Reciclado de Endpoints

Riesgo: Comportamiento no deseado en tráfico East-West multiclúster cuando cambia la IP de pods backend remotos (pod churn), generando requests colgadas o degradación de disponibilidad.

Probabilidad: Media
Impacto: Alto

Mitigación:

- Pruebas obligatorias de "pod churn cross-cluster" en PoC, staging y preproducción
- Sondas sintéticas continuas para detectar colgado de requests entre clusters
- Validación de políticas de timeout, reconnect y drenaje de conexiones en el data plane
- Criterio de no-go-live si el escenario no pasa de forma consistente
- Runbook operativo de contingencia y rollback documentado

R3: Integración con Infraestructura Existente

Riesgo: Dificultades en integración con F5, DNS, identity providers, sistemas de observabilidad.

Probabilidad: Media
Impacto: Medio

Mitigación:

- Validación de integraciones en POC
- Involucrar equipos de infraestructura temprano
- Documentación detallada de integraciones
- Testing exhaustivo de integraciones
- Plan de contingencia para cada integración

R4: Curva de Aprendizaje del Equipo

Riesgo: El equipo puede tener dificultades aprendiendo la nueva tecnología, impactando velocidad de migración.

Probabilidad: Alta
Impacto: Medio

Mitigación:

- Training temprano del equipo
- Documentación completa
- Soporte de vendor durante migración
- Centro de excelencia interno
- Pair programming y knowledge sharing

Riesgos Operacionales

R5: Failover Automático No Funcional

Riesgo: El failover automático puede no funcionar correctamente en producción, causando downtime.

Probabilidad: Baja
Impacto: Crítico

Mitigación:

- Testing exhaustivo de failover en POC
- Pruebas de failover en ambiente de staging
- Monitoreo continuo de health checks
- Procedimientos manuales de failover como backup
- DR drills regulares

R6: Desincronización entre Clusters

Riesgo: Configuraciones pueden desincronizarse entre clusters, causando inconsistencias.

Probabilidad: Baja
Impacto: Alto

Mitigación:

- Configuración declarativa (GitOps)
- Sincronización automática desde control plane
- Validación de configuraciones antes de aplicar
- Monitoreo de drift de configuración
- Alertas automáticas de desincronización

R7: Soporte y Escalación

Riesgo: Soporte del vendor puede ser insuficiente o lento, impactando resolución de incidentes.

Probabilidad: Media
Impacto: Medio

Mitigación:

- SLA claramente definido en contrato
- Equipo interno capacitado para troubleshooting básico
- Documentación de troubleshooting común
- Escalación procedures documentados
- Vendor relationship management

Riesgos de Negocio

R8: Timeline de Migración Extendido

Riesgo: La migración puede tomar más tiempo del planeado, acercándose al EOL de 3scale.

Probabilidad: Media
Impacto: Alto

Mitigación:

- Plan de migración realista con buffers
- Priorización clara de APIs críticas
- Recursos dedicados al proyecto
- Monitoreo continuo del progreso
- Escalación temprana de problemas

R9: Costos Exceden Presupuesto

Riesgo: Costos de implementación u operación pueden exceder el presupuesto aprobado.

Probabilidad: Media
Impacto: Medio

Mitigación:

- Fixed pricing negociado para tráfico interno
- Presupuesto detallado con contingencia
- Monitoreo continuo de costos
- Optimización continua de recursos
- Revisión regular de costos vs presupuesto

R10: Vendor Lock-in

Riesgo: Dependencia excesiva del vendor puede limitar opciones futuras.

Probabilidad: Baja
Impacto: Medio

Mitigación:

- Base open source y estándares abiertos (Cilium/eBPF + Envoy/Gateway API)
- Configuraciones declarativas (portables)
- Gateway API como estándar (evita lock-in)
- Documentación de estrategias de salida
- Evaluación periódica de alternativas

Matriz de Riesgos

Riesgo	Probabilidad	Impacto	Prioridad	Mitigación
R1: Complejidad Migración	Alta	Alto	🔴 Crítica	Scripts automatizados, migración gradual
R2: Performance Inferior	Media	Alto	🔴 Crítica	Pruebas exhaustivas, baseline documentado
R3: Integración Infraestructura	Media	Medio	🟡 Alta	Validación en POC, testing exhaustivo
R4: Curva Aprendizaje	Alta	Medio	🟡 Alta	Training temprano, documentación
R5: Failover No Funcional	Baja	Crítico	🔴 Crítica	Testing exhaustivo, DR drills
R6: Desincronización	Baja	Alto	🟡 Alta	GitOps, monitoreo de drift
R7: Soporte Insuficiente	Media	Medio	🟡 Alta	SLA definido, equipo capacitado
R8: Timeline Extendido	Media	Alto	🔴 Crítica	Plan realista, monitoreo continuo
R9: Costos Exceden	Media	Medio	🟡 Alta	Fixed pricing, monitoreo de costos
R10: Vendor Lock-in	Baja	Medio	🟢 Media	Open source, configuraciones portables
R11: Inestabilidad Cross-Cluster	Media	Alto	🔴 Crítica	Pod churn tests, no-go-live, runbooks

9. Conclusiones y Próximos Pasos

Estado Actual del Proyecto

El proyecto ha evolucionado desde un simple reemplazo de 3scale hacia una modernización arquitectónica integral. La evaluación de múltiples vendors ha revelado la necesidad de una solución que combine:

- **Service Mesh** para tráfico interno (East-West)
- **API Gateway** para tráfico externo (North-South)
- **Arquitectura multiclúster** con failover automático
- **Fixed pricing** para sostenibilidad financiera

Decisión Arquitectónica Tomada

La **arquitectura objetivo no cambia en su modelo**, pero sí en el componente East-West seleccionado:

- **Service Mesh Sidecarless East-West: Cilium Mesh (Isovalent Enterprise)**
- **API Gateway North-South:** Preferencia por **Gloo Gateway**
- **Arquitectura multiclúster** con failover automático
- **Fixed pricing** para sostenibilidad financiera

Camino Transitado con Vendor para Cerrar la Decisión

- Se ejecutaron PoCs de Ambient Mesh multiclúster en OpenShift on-prem y luego en OpenShift sobre EC2 para descartar sesgo de entorno.
- En ambos casos se reprodujo la misma falla: tráfico `cluster1 -> cluster2` queda colgado cuando se recicla el pod backend remoto y cambia su IP.
- La causa raíz validada fue reutilización de conexión TCP stale en `ztunnel` (HBONE `:15008`) hacia IP antigua del pod.
- Se aplicó el workaround de reinicio de East-West gateway (`ztunnel`), confirmando recuperación temporal pero no solución de fondo.
- Se ejecutaron además escenarios sugeridos por vendor (incluyendo failover del workshop público) y no reprodujeron la falla, por tratarse de un patrón de tráfico distinto.
- Conclusión: el issue no es dependiente del entorno (on-prem/AWS), sino del patrón de tráfico cross-cluster con rotación de endpoints.

Estado de Alternativas

- **Istio Ambient Mesh (Solo.io / RHOSM):** Descartado para el dominio East-West de esta iniciativa.
- **Cilium Mesh (Isovalent Enterprise):** Seleccionado para East-West multiclúster.
- **API Gateway North-South:** Continúa la línea de adopción con Gloo Gateway.

Arquitectura Objetivo

Tráfico North-South (3 capas):

- DMZ → API Gateway (B2B) → Service Mesh → Backend Services

Tráfico East-West:

- Service Mesh directo (sin hair-pinning)

Despliegue:

- Gloo Gateway on-demand en namespaces que lo requieran
- Service Mesh Sidecarless desplegado en todos los clusters (Cilium Mesh Enterprise para East-West)

Próximos Pasos Críticos

1. **Cerrar frente contractual y soporte enterprise de Cilium Mesh (Isovalent)**
   - Licenciamiento, soporte y modelo operativo
   - Definición de ownership entre Platform, Networking y DevOps
2. **Diseñar arquitectura detallada de implementación**
   - Integración Cilium Mesh East-West + API Gateway North-South
   - Runbooks de operación y contingencia
   - Guardrails de seguridad y observabilidad
3. **Ejecutar PoC de hardening y performance sobre stack seleccionado**
   - Desplegar en 2 clusters

    - Validar funcionalidad, performance e integraciones
    - Incluir pruebas obligatorias de pod churn cross-cluster
4. **Planificación de migración**
    - Inventario y priorización de APIs
    - Estrategia detallada de transición desde 3scale
    - Automatización de migración y validación
5. **Implementación piloto y migración gradual**
    - Migración de subset inicial
    - Ajustes operativos
    - Escalamiento progresivo al resto de APIs

Timeline Crítico

- **Q1 2026:** Cierre contractual y diseño detallado
- **Q2 2026:** PoC de hardening/performance e implementación piloto
- **Q3-Q4 2026:** Migración gradual
- **Q4 2026:** Go-live completo
- **Mediados 2027:** 3scale EOL (deadline absoluto)

Documento preparado: Febrero 2026
Próxima revisión: Post-PoC de hardening de Cilium Mesh Enterprise